

HAWASSA UNIVERSITY

Institute of Technology

Department of Computer Science

Research Methods in Computer Science

Artificial Intelligence / Cybersecurity

**Evaluating the Robustness of Large
Language Models (LLMs) Against Prompt
Injection Attacks in Automated Customer
Service Systems**

Name: Abdurehim Mohammed

ID: 1034/16

Submitted To: Birehane D.

Submission Date: May 1 , 2026

CONTENTS

1	Introduction	2
2	Objectives	3
3	Literature Review and Synthesis Matrix	4
3.1	Synthesis Matrix	4
3.2	Synthesis and Research Gap	5
4	Proposed Methodology	6
4.1	System Architecture	6
4.2	Tool Justification	7
4.3	Data Strategy	8
4.4	Evaluation Metrics	8
4.5	Software Development Lifecycle	8
5	Significance and Impact	9
6	Ethical Considerations	9
7	Project Timeline — Gantt Chart	10
8	Conclusion	11

INTRODUCTION

The emergence of Large Language Models (LLMs) as the engine behind automated customer service platforms represents one of the most consequential shifts in how organizations interact with their customers. Systems powered by models such as GPT-4, Claude, LLaMA, and Mistral are now deployed at scale across banking, telecommunications, e-commerce, and healthcare — handling sensitive queries, processing transactions, and managing account information around the clock with minimal human oversight. This adoption has driven measurable gains in operational efficiency: reduced staffing costs, extended service hours, and faster response times have made LLM-powered agents an attractive investment for organizations of every scale.

However, this rapid deployment has outpaced the development of the security frameworks needed to protect these systems. LLMs were not designed with adversarial robustness as a primary objective; they were optimized for language understanding and generation. This fundamental design orientation creates a security surface that conventional cybersecurity tools were not built to address. Traditional defenses — firewalls, intrusion detection systems, input validation rules — operate on structured data and known attack signatures. They have no mechanism for detecting a threat that arrives as a grammatically correct, contextually plausible sentence.

It is within this context that prompt injection has emerged as a critical and growing vulnerability class. Recognized by OWASP as the top risk in its LLM Application Security Top 10 (2023), prompt injection attacks exploit the inability of current transformer-based architectures to maintain a reliable trust boundary between system-level instructions and user-provided inputs. Because both types of content are processed as tokens within the same context window, a sophisticated user can craft an input that the model interprets as a higher-priority instruction — overriding safety constraints, extracting protected information, or redirecting system behavior in ways that violate organizational policies and customer trust.

Customer service deployments are particularly attractive targets for this class of attack. These systems routinely handle sensitive personal and financial data, operate with deeply embedded business logic in their system prompts, and maintain persistent conversation histories that sophisticated attackers can manipulate across multiple turns. Despite the severity of this exposure, the research community has not yet produced a standardized, domain-specific framework for evaluating or defending against prompt injection in this context. This proposal addresses that gap.

Problem Statement

Ideal

An LLM-powered customer service system should maintain an inviolable separation between trusted system instructions and untrusted user inputs. It should reliably detect and reject adversarial attempts to override its behavioral constraints — regardless of phrasing, encoding, or multi-turn distribution — while security teams have access to standardized, domain-specific evaluation tools to audit robustness before deployment and continuously during operation.

Reality

Current LLMs process system prompts and user inputs as tokens within the same context window, with no architectural trust hierarchy separating them. Adversarially crafted inputs routinely override system directives, elicit sensitive data, and impersonate system authority across all leading model families. No validated, customer-service-specific adversarial evaluation benchmark currently exists, leaving organizations unable to measure or manage their exposure. Existing defenses — keyword blacklists, prompt delimiters, and output classifiers — have not been rigorously tested against sophisticated, multi-variant injection campaigns in domain-realistic settings, and their failure modes remain poorly characterized.

Consequence

Successful prompt injection attacks against deployed customer service LLMs result in unauthorized disclosure of personal and financial data, fraudulent execution of account actions, regulatory penalties under data protection law, and erosion of consumer trust. These consequences are disproportionately severe in developing countries and emerging markets where digital consumer protection frameworks remain immature, recourse mechanisms are limited, and the consequences of a breach fall most heavily on users with the least institutional support.

OBJECTIVES

General Objective: To design, implement, and empirically validate a domain-specific adversarial evaluation framework that systematically assesses the robustness of LLMs against prompt injection attacks in automated customer service environments, and to derive and test a layered defense architecture that measurably reduces attack success rates without compromising legitimate system functionality.

Specific SMART Objectives:

SO1: Construct and manually validate a taxonomy of 120+ prompt injection test cases — spanning direct, indirect, multi-turn, and obfuscation-based

attack variants — by end of Month 3 (Specific, Measurable by case count and validation rate, Achievable, Relevant, Time-bound).

SO2: Evaluate the Attack Success Rate (ASR) and Behavioral Compliance Score (BCS) of four leading LLMs (GPT-4o, Claude 3.5, LLaMA 3, Mistral 7B) under the full test suite within a controlled Simulated Customer Service Evaluation Environment (SCSEE) by end of Month 5.

SO3: Implement and compare four defense mechanisms — input sanitization, instruction hierarchy enforcement, BERT-based output guardrail, and RAG injection filtering — measuring Defense Bypass Rate (DBR) and False Positive Rate (FPR) for each mechanism by end of Month 7, with a target FPR below 8%.

LITERATURE REVIEW AND SYNTHESIS MATRIX

Synthesis Matrix

Author & Source	Method	Dataset	Key Metric	Limitation
Perez & Ribeiro (2022) [3] <i>ACM CCS</i>	Role reassignment and goal hijacking against InstructGPT; manual adversarial prompt construction	Custom crafted prompts (n=50 templates)	ASR: 72% with no defenses	Abstract NLP only; no domain context, session modeling, or defense evaluation
Greshake et al. (2023) [1] <i>IEEE S&P</i>	Indirect injection via malicious content in web pages and documents retrieved by LLM agents	Custom indirect injection set using live retrieval chains	Data exfiltration: 68%; code execution success: 91%	No defense evaluation; no customer service business logic or session context

Author & Source	Method	Dataset	Key Metric	Limitation
Liu et al. (2023) [2] <i>ACM CCS</i>	Systematic taxonomy of 5 injection categories across 10 LLM-integrated applications	Self-constructed benchmark (10 real-world apps)	ASR range: 15%–85% by model and attack type	No defense evaluation; customer service context absent; no multi-turn attacks
Toyer et al. (2024) [4] <i>IEEE Trans. Dep. Sec. Comp.</i>	Game-based crowdsourced injection elicitation (Tensor Trust) with labeled outcomes	126,000+ labeled attack interactions	Smaller models 3× more vulnerable than larger models	Game context lacks ecological validity; no real data flows or business logic
Yi et al. (2025) [5] <i>Comput. & Security</i>	Benchmarking of 5 defense strategies across 8 LLMs using standardized injection suite	PromptBench + custom extension (n=240 cases)	Best defense: ASR 67%→22%; FPR=11%	11% FPR commercially unacceptable; no multi-turn or CS-domain scenarios tested

Synthesis and Research Gap

The five studies above collectively define the current state of the art in prompt injection security and, taken together, reveal a precise and significant gap that this research is designed to close. Perez and Ribeiro [3] established the foundational vulnerability: instruction-tuned LLMs cannot reliably separate adversarial user inputs from legitimate system instructions, producing an Attack Success Rate of 72% under undefended conditions. Their work, however, operated in a context-free setting with a small manually crafted prompt set — a methodology that cannot capture the threat model of a deployed customer service system where real account data flows through conversations and attackers may sustain campaigns across multiple interaction turns.

Greshake et al. [1] extended the threat surface critically by demonstrating that injection does not require direct user interaction. Malicious instructions embedded within documents retrieved by an LLM agent — a design pattern central to RAG-based customer service deployments — can compromise system behavior without any visible at-

tacker input. This indirect attack vector is of particular relevance to the deployment context this research targets, yet no defensive evaluation accompanied their study. Liu et al. [2] contributed the most comprehensive attack taxonomy, confirming that ASR varies dramatically (15%–85%) by model architecture and attack category, reinforcing the importance of model-specific evaluation — but again without defense evaluation or domain specificity.

The Tensor Trust dataset from Toyer et al. [4] provides the largest empirical injection resource yet assembled, but the game setting that generated it lacks the ecological validity of a real customer service deployment: there are no sensitive account records to protect, no business policies to bypass, and no economic incentives shaping attacker behavior. Most critically, Yi et al. (2025) [5] — the closest existing work to a defense evaluation framework — achieved an 11% False Positive Rate under the best-performing defense combination. One in nine legitimate customer queries incorrectly blocked represents a commercially unacceptable outcome for any production deployment. The gap this research fills is therefore specific and well-defined: no existing study has simultaneously evaluated prompt injection robustness within a realistic customer service architecture, measured defenses against domain-specific attack scenarios, and produced a framework achieving low ASR at a commercially viable FPR below 8%. This project addresses all three dimensions.

PROPOSED METHODOLOGY

System Architecture

The research will be conducted within a purpose-built Simulated Customer Service Evaluation Environment (SCSEE) — a four-layer modular platform designed to replicate the functional architecture of a real-world LLM-powered customer service deployment while providing the instrumentation required for rigorous adversarial testing:

- **Customer Interface Layer (React.js):** A browser-based chat portal replicating the user-facing input surface. Handles session initiation, conversation history rendering, and user input capture — the primary interface through which all injection test cases are submitted.
- **Orchestration Middleware Layer (Python / FastAPI):** The core experimental control plane. Manages LLM API routing, enforces system prompt configurations, activates and deactivates defense mechanisms, and logs all experimental events in structured JSON format.

- **LLM Integration Layer (OpenAI API / Anthropic API / Ollama):** Standardized API connections to GPT-4o, Claude 3.5 Sonnet, LLaMA 3 8B, and Mistral 7B. All models receive identically structured prompts to ensure valid cross-model comparison.
- **Logging and Analysis Layer (PostgreSQL / Jupyter):** Schema-validated event storage with a Python analysis module computing all robustness metrics and producing comparative visualizations for reporting.

Tool Justification

Tool	Alternative Considered	Justification
Python 3.11 + FastAPI	Node.js / Express	Native async minimizes latency during high-volume test runs; direct access to full ML ecosystem (transformers, scikit-learn, sentence-transformers) without bridging layers.
Ollama (local LLM inference)	HuggingFace Transformers direct	Unified model-agnostic API wrapper eliminates per-model configuration differences that would introduce confounds across open-source model comparisons.
sentence-transformers (all-mpnet-base-v2)	TF-IDF / BM25	Dense semantic embeddings detect paraphrased and obfuscated injection patterns that lexical similarity measures miss — critical for obfuscation attack detection.
Fine-tuned BERT output classifier	Regex rule-based classifier	BERT recognizes policy violations through learned semantic patterns; regex-based rules fail against novel phrasings not present in the predefined rule set.
Garak LLM vulnerability scanner	Manual test-case execution only	Automates structured probe suites at throughput scale impractical through manual submission; enables fully reproducible, version-controlled experimental testing.

Data Strategy

Dataset Sources: The attack test suite is seeded from PromptBench (IEEE Data-Port), the Tensor Trust Dataset, and the Garak probe library, and extended with 60 novel customer-service-specific injection variants — covering policy bypass, unauthorized refund manipulation, account privilege escalation, and multi-turn context poisoning — each manually validated by the research team. All customer profile data used as the protected target is fully synthetic, generated via the Python Faker library, eliminating all personal data exposure risk.

Pre-processing Pipeline: (1) Unicode normalization (NFC form) to neutralize homoglyph substitution attacks while preserving raw inputs for analysis. (2) Semantic embedding via all-mpnet-base-v2 for similarity-based injection detection. (3) Attack label assignment by category, expected behavior, and severity tier. (4) Schema validation of all logged experimental events before database insertion to ensure analytical integrity.

Evaluation Metrics

Metric	Symbol	Formula	Target
Attack Success Rate	ASR	$\frac{SuccessfulAttacks}{TotalAttempts} \times 100\%$	< 30%
Defense Bypass Rate	DBR	$\frac{BypassingAttacks}{TotalPost-Defense} \times 100\%$	< 25%
False Positive Rate	FPR	$\frac{Legit.QueriesBlocked}{TotalLegit.Queries} \times 100\%$	< 8%
Behavioral Comp.	BCS	$\frac{CompliantResponses}{TotalResponses} \times 100\%$	> 90%
Data Leakage Rate	DLR	$\frac{PIIResponses}{TotalResponses} \times 100\%$	< 2%
Defense Latency	DOL	Mean Latency (ON - OFF)	< 400 ms

Statistical significance of differences across model and defense conditions is assessed using paired Wilcoxon signed-rank tests ($\alpha = 0.05$, Bonferroni-corrected for multiple comparisons across four models).

Software Development Lifecycle

The project follows a hybrid Agile-Scrum and scientific experimental design methodology across six phases spanning 24 weeks. Agile two-week sprints govern SCSEE software development; a pre-registered experimental protocol governs adversarial evaluation phases — ensuring engineering flexibility while maintaining the scientific rigor required for reproducible results. All experimental parameters are documented publicly before data collection begins.

SIGNIFICANCE AND IMPACT

The significance of this research extends across three interconnected dimensions. At the technical level, it delivers the first domain-specific adversarial evaluation framework for LLMs in customer service environments — reusable infrastructure for researchers and security practitioners. The SCSEE, standardized attack taxonomy, and comparative robustness benchmarks will constitute an open-source resource that other investigators can build upon and extend to adjacent domains.

At the organizational level, the findings directly benefit the growing number of companies that have deployed or are planning to deploy LLM-powered customer service agents — particularly those in developing countries that lack in-house AI security expertise. A publicly available evaluation framework and security guideline document gives these organizations practical, evidence-based tools to assess and improve their defenses without requiring deep specialist knowledge or expensive proprietary infrastructure.

At the societal level, the research contributes to digital trust — the foundational condition for the economic and social benefits that AI-powered services promise. Where digital financial and service infrastructure is expanding rapidly and consumer protection frameworks are still developing, strengthening the security of LLM deployments has direct implications for the welfare of millions of users whose sensitive data and financial transactions depend on the integrity of these systems.

ETHICAL CONSIDERATIONS

GDPR and Data Privacy: All experimental data is fully synthetic. No real personal, financial, or health information is processed at any stage of the research. The SCSEE operates in a network-isolated environment, and commercial API calls comply with provider data usage terms. A formal Data Protection Impact Assessment (DPIA) aligned with GDPR Article 25 (Privacy by Design) will be completed before experiments begin, alongside formal institutional Research Ethics Committee clearance.

Algorithmic Bias: A pre-registered experimental protocol — documenting all test cases, defense configurations, and evaluation metrics before data collection begins — prevents evaluation bias from inadvertently favoring specific model architectures. Standardized API call structures ensure all models receive identical context configurations. The attack test suite is independently reviewed for linguistic and stylistic bias to ensure that attack difficulty is not correlated with writing characteristics that may differentially affect models trained on different corpora.

Environmental Impact: Open-source local inference models (LLaMA 3, Mistral via Ollama) are prioritized throughout the experimental design to minimize commercial API compute consumption. Experimental runs are batched to maximize GPU utilization efficiency. The CodeCarbon Python library will track and report the study’s total carbon footprint in the final published work, supporting the emerging norm of environmental transparency in AI research.

Responsible Disclosure: Any novel attack technique developed during the research that is found to be highly effective against specific currently deployed systems will be handled under a 90-day responsible disclosure protocol, notifying relevant model providers before public publication. Research findings will be communicated at the level of principles, categories, and defensive implications — not as ready-made exploitation tools.

PROJECT TIMELINE — GANTT CHART

Phase – Key Activities and Deliverables	Weeks	M1	M2	M3	M4	M5	M6	M7	M8	M9
Ph.1: Literature Review & Gap Analysis	1–4									
Ph.2: SCSEE Environment Setup	5–8									
Ph.3: Attack Suite Development	9–12									
Ph.4: Baseline Vulnerability Evaluation	13–20									
Ph.5: Defense Mechanism Implementation & Testing	21– 24									
Ph.6: Layered Defense Optimization	25–28									
Ph.7: Analysis, Documentation & Final Writing	29–36									

Legend: Active phase durations are shaded. Each phase includes deliverables and outputs that feed into subsequent phases. Phases 4–6 include iterative feedback loops where findings from evaluation phases directly inform defense implementation priorities.

Key Milestones: **Month 3:** Attack taxonomy finalized | **Month 5:** Baseline evaluation complete | **Month 7:** All defenses evaluated | **Month 9:** Final framework and dissertation submitted

CONCLUSION

Prompt injection represents a cybersecurity threat unlike any that conventional tools were designed to address — it operates in the same natural language that LLMs are built to understand and follow, making it invisible to signature-based defenses and architecturally difficult to contain. As organizations continue to deploy LLM-powered systems in customer service roles that handle sensitive data and execute consequential actions, the absence of validated, domain-specific security evaluation frameworks represents a growing and increasingly costly risk.

This research offers a rigorous and practically grounded response. The three expected outcomes are: (1) a validated open-source adversarial evaluation framework (SC-SEE) providing both researchers and industry practitioners with a reproducible robustness assessment tool tailored to the customer service deployment context; (2) a comparative robustness benchmark — the first domain-specific ASR and BCS analysis across four leading LLMs under standardized injection conditions; and (3) a layered defense framework empirically validated to achieve at least 65% ASR reduction while maintaining a commercially viable False Positive Rate below 8%. Together, these contributions move the field from theoretical concern to actionable, deployable security — enabling organizations to build LLM-powered customer service systems that are not only capable and efficient, but genuinely trustworthy.

REFERENCES

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injections. In *Proc. IEEE Symposium on Security and Privacy (S&P)*, pages 1–16, San Francisco, CA, USA, May 2023.
- [2] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu. Prompt injection attack against llm-integrated applications. In *Proc. ACM Conf. on Computer and Communications Security (CCS)*, pages 1–18, Copenhagen, Denmark, Nov. 2023.
- [3] F. Perez and I. Ribeiro. Ignore previous prompt: Attack techniques for language models. In *Proc. ACM Conf. on Computer and Communications Security (CCS)*, pages 1–12, Los Angeles, CA, USA, Nov. 2022.
- [4] S. Toyer, O. Watkins, E. A. Mendes, J. Svegliato, L. Bailey, T. Wang, and A. Dragan. Tensor trust: Interpretable prompt injection attacks from an online game. *IEEE Trans. Dependable Secure Comput.*, 21(3):1450–1465, May–Jun. 2024.
- [5] X. Yi, Z. Chen, M. Backes, Y. Shen, and Y. Zhang. Benchmarking prompt injection defenses in large language model applications. *Comput. Secur.*, 142:1–18, May 2025.